Algoritmos de ordenamientos

# Programación II

15

Octubre

UNRN

Universidad Nacional
de **Río Negro**

r27

# Programación I

## Algoritmos de ordenamientos

Universidad Nacional de Río Negro | Ingeniería en Sistemas | Ciclo 2026

**Prof. Daniel Teira/Diego Diaz**

Sede Bariloche

# Contenidos de la clase

- **Recursividad**
- **Definiciones**
- **Algoritmos de ordenamiento N: N2 Insertion, Selecction, Bubble. Cálculo de  complejidad.**
- **Algoritmos de ordenamiento eficientes: quicksort y mergeSort.**
- **Comparación con los ineficientes,**

# Recursividad

**Definición informal: La recursión es el acto de resolver un problema delegando una versión más pequeña de ese mismo problema a ti mismo, hasta llegar a una versión tan ridículamente simple que la respuesta es obvia.**

En programación, la recursión ocurre cuando **una función se llama a sí misma** dentro de su propio bloque de ejecución.

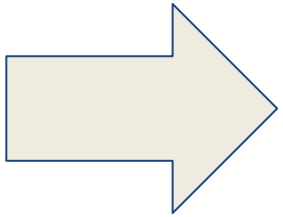
Para que un algoritmo recursivo sea matemáticamente válido y no bloquee la computadora en un bucle infinito, debe poseer estrictamente dos componentes estructurales:

1. **El Caso Base (La condición de corte):** Es la condición simple que detiene la recursividad. Representa el problema más pequeño posible que se puede resolver directamente sin volver a llamar a la función. (En la metáfora, es la primera persona de la fila).
2. **El Caso Recursivo (El paso inductivo):** Es la parte de la función que reduce el tamaño del problema actual y realiza una nueva llamada a sí misma con esos datos más pequeños, acercándose inexorablemente hacia el caso base.

# Recursividad

Caso ejemplo el factorial

```
int factorial(int n) {
    // 1. El Caso Base: Si llegamos a 1 (o 0), sabemos la respuesta directa.
    if (n <= 1) {
        return 1;
    }
    // 2. El Caso Recursivo: Multiplicamos 'n' por el factorial del número anterior.
    else {
        return n * factorial(n - 1);
    }
}
```



```
funcion( valor ){
   if ( valor == 0 )
   {
      return value
   }
   else
   {
      return funcion( valor - 1 )
   }
}
```

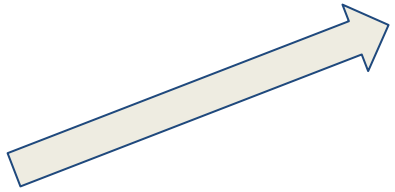
Caso base

Llamada recursiva

Estructura base

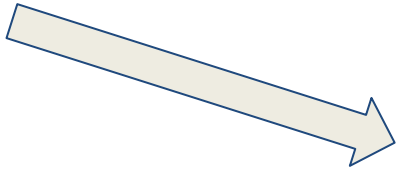
# Recursividad

## Ventajas



El código es intuitivo en su proposito.
La división de un problema en subproblemas más simples es directa.

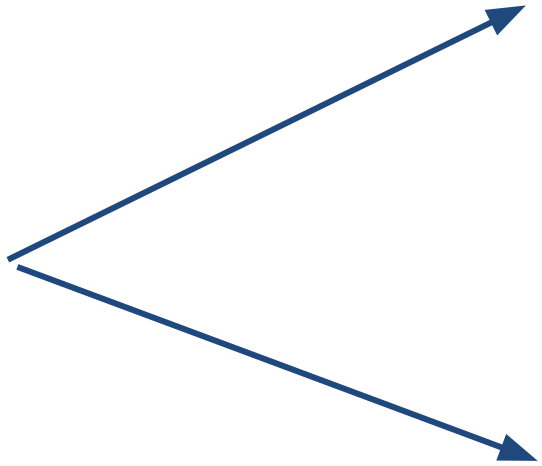
## DesVentajas



Consumo de memoria elevado[1]
¡Es más fácil caer en un lazo infinito!
Son generalmente más lentas[2]
No hay forma de manipular la repetición

# Recursividad

Cuando conviene usarlo?



**Cuándo SÍ conviene usar Recursión**

Debes elegir la recursión cuando el problema tiene una **estructura naturalmente jerárquica, de árbol o auto-similar**.

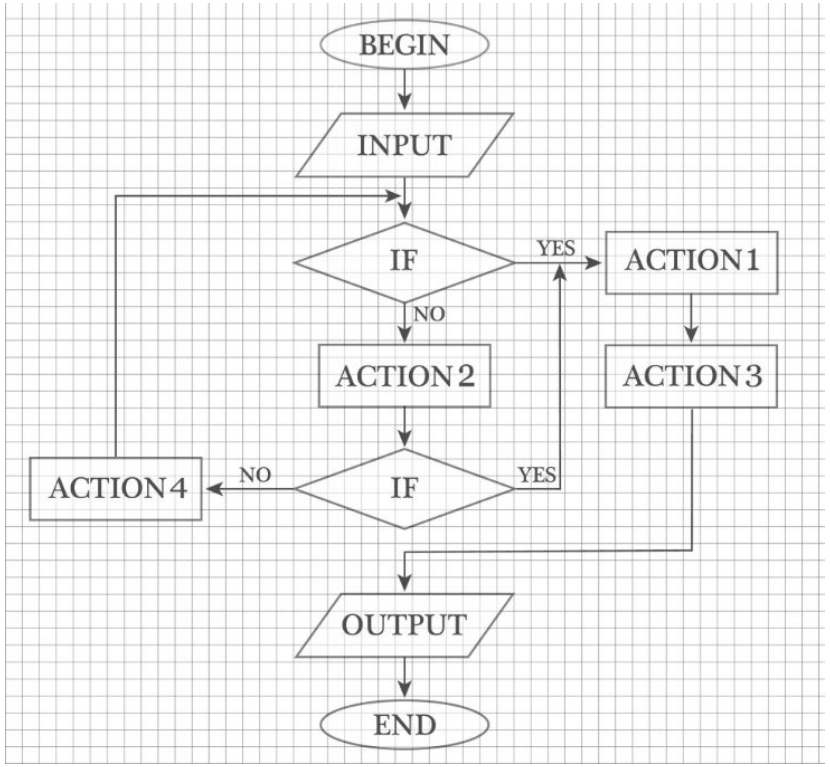
**Cuándo NO conviene usar Recursión (Usa Iteración)**

Debes huir de la recursión cuando el problema es **estrictamente secuencial o lineal**, o cuando estás programando para hardware con recursos muy limitados

# Definición de algoritmo:

Un algoritmo es, en su definición más fundamental, un **conjunto finito, ordenado y sin ambigüedades de instrucciones** diseñado para resolver un problema específico o realizar una tarea.

*Funciona como una "receta" lógica: toma un estado inicial o una serie de datos de entrada (input), los procesa paso a paso mediante reglas matemáticas o lógicas, y produce un resultado o estado final (output).*



# Definición de algoritmo:

## Características de un Algoritmo

Para que un conjunto de instrucciones sea considerado formalmente un algoritmo, debe cumplir con cinco propiedades matemáticas (definidas históricamente por Donald Knuth):

1. **Finitud:** Debe terminar siempre después de un número finito de pasos. No puede quedarse atrapado en un bucle infinito de forma no intencionada.
2. **Precisión (Definibilidad):** Cada paso debe estar definido de manera exacta y rigurosa. No puede haber ambigüedad ni depender de la interpretación subjetiva de quien lo ejecuta o de la computadora.
3. **Entrada (Input):** Recibe cero o más cantidades de datos externos antes de comenzar a ejecutarse.
4. **Salida (Output):** Produce al menos un resultado o efecto medible, el cual tiene una relación específica y predecible con la entrada.
5. **Eficacia:** Todas las operaciones requeridas deben ser lo suficientemente básicas y sencillas como para poder ser ejecutadas de manera exacta y en un lapso de tiempo finito.

**Algoritmo de Euclides (Matemático):** Uno de los más antiguos conocidos. Se utiliza para encontrar el máximo común divisor (MCD) de dos números. *Regla:* Divide el número mayor por el menor; si hay un resto, reemplaza el número mayor con el menor y el menor con el resto. Repite hasta que el resto sea cero.

**Búsqueda Binaria (Computacional):** Encuentra un elemento específico en una lista *que ya está ordenada*. *Regla:* Compara el elemento buscado con el valor central de la lista; si es menor, descarta la mitad superior y repite el proceso con la mitad inferior, reduciendo el espacio de búsqueda a la mitad en cada paso.

**Algoritmo de Dijkstra (Redes/Geometría):** Calcula la ruta más corta entre dos puntos en un grafo o mapa. Es el algoritmo fundacional detrás de cómo un GPS calcula tu ruta para llegar a casa o cómo viajan los paquetes de datos por internet.

# Formas de representar algoritmos:

1. Pseudocódigo
2. Diagramas de flujo
3. Código en un lenguaje específico

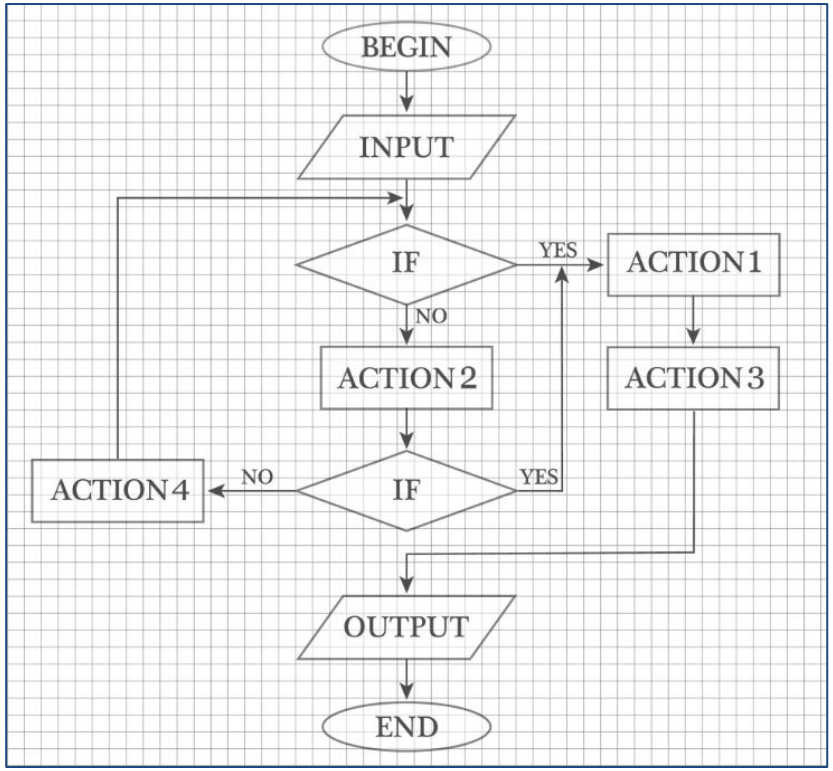
```
Algoritmo EncontrarTemperaturaMaxima
Entrada: Un arreglo 'lecturas' de tamaño 'N'
Salida: El valor máximo registrado

Inicio
    Si N == 0 entonces
        Retornar error (arreglo vacío)
    Fin Si

    // Asumimos que el primer elemento es el
mayor inicialmente
    maximo = lecturas[0]

    // Recorremos el resto del arreglo
    Para i = 1 hasta N-1 hacer
        Si lecturas[i] > maximo entonces
            maximo = lecturas[i] //
Actualizamos el nuevo máximo
        Fin Si
    Fin Para

    Retornar maximo
Fin
```




```c
#include <stdio.h>

// Función que implementa el algoritmo
float encontrar_temperatura_maxima(float lecturas[], int n) {
    // 1. Manejo del caso base (arreglo vacío)
    if (n <= 0) {
        return -999.0; // Código de error representativo
    }

    // 2. Inicialización
    float maximo = lecturas[0];

    // 3. Ciclo de recorrido y comparaciones
    for (int i = 1; i < n; i++) {
        if (lecturas[i] > maximo) {
            maximo = lecturas[i]; // Se encontró un nuevo valor pico
        }
    }

    // 4. Salida
    return maximo;
}

int main() {
    // Datos de prueba simulando entradas de telemetría térmica
    float sensores_temp[] = {15.4, 22.1, 18.9, 25.6, 20.2};

    // Calculamos dinámicamente la longitud del arreglo
    int num_lecturas = sizeof(sensores_temp) / sizeof(sensores_temp[0]);

    // Ejecución del algoritmo
    float max_temp = encontrar_temperatura_maxima(sensores_temp, num_lecturas);

    printf("La temperatura máxima registrada es: %.2f grados\n", max_temp);

    return 0;
}
```

# Complejidad

**La complejidad: "*cómo escala el consumo de recursos de un algoritmo a medida que aumenta el tamaño de los datos de entrada*"** y no es qué tan difícil es escribir o entender un fragmento de código

Se evalúa fundamentalmente en dos dimensiones, las cuales suelen competir entre sí (a menudo, para ganar velocidad, se debe sacrificar memoria):

**Complejidad Temporal:** Cuántas operaciones elementales (ciclos de procesamiento) requiere el algoritmo para finalizar.

**Complejidad Espacial:** Cuánta memoria RAM adicional necesita el algoritmo para guardar variables temporales, pilas de llamadas o estructuras de datos auxiliares durante su ejecución.

# La Notación Big O (“O grande”)

En ciencias de la computación, rara vez medimos la complejidad en "segundos" o "bytes" absolutos, porque el hardware cambia constantemente. En su lugar, utilizamos la **Notación Big O**, un estándar matemático que describe el límite superior asintótico. En términos simples: describe el **peor escenario posible** de rendimiento cuando “**N**” tiende al infinito.

categorías de complejidad más comunes

***O*(1) - Constante:** El tiempo de ejecución no cambia, sin importar si *N* es 10 o 10 millones.

- *Ejemplo en C:* Acceder a una posición específica de un arreglo en memoria mediante un puntero o su índice (arreglo[5]).

***O*(log N) - Logarítmica:** El algoritmo divide el conjunto de datos en cada iteración. A medida que los datos crecen masivamente, el tiempo apenas sube.

- *Ejemplo:* Búsqueda binaria en un arreglo previamente ordenado.

***O*(N) - Lineal:** El tiempo crece en proporción directa e idéntica a la cantidad de datos. Si los datos se duplican, el tiempo de ejecución se duplica.

- *Ejemplo:* Recorrer un arreglo con un simple bucle for buscando un valor, o calcular el máximo de una serie de lecturas de sensores.

***O*(N log N) - Linealítmica:** Es la cota de eficiencia óptima teórica para ordenar conjuntos de datos de forma comparativa.

- *Ejemplo:* Algoritmos como Quicksort y Mergesort.

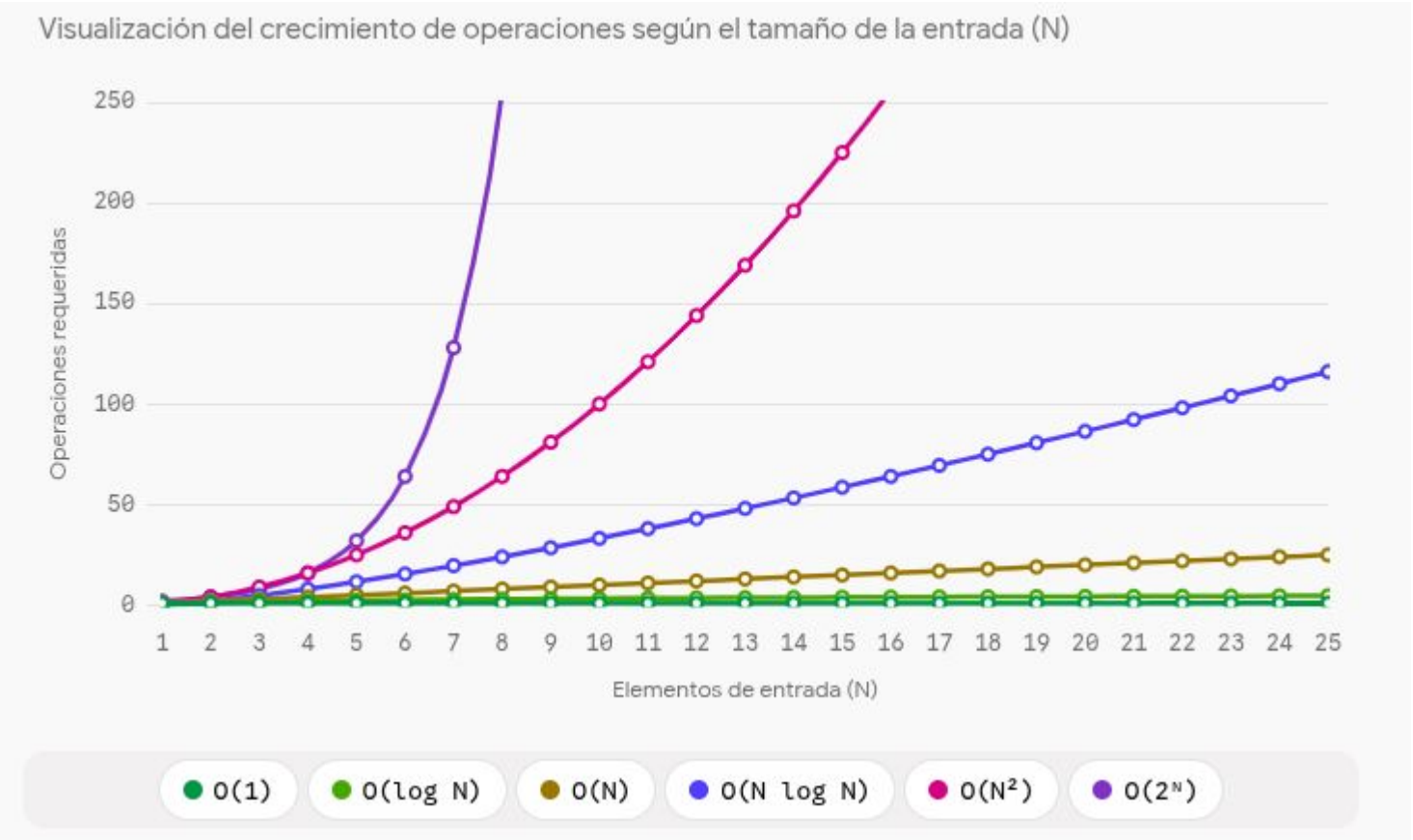
***O*(N2) - Cuadrática:** El tiempo crece al cuadrado respecto a los datos, comúnmente debido a la presencia de bucles anidados. Peligroso para grandes conjuntos de información.

- *Ejemplo:* Los algoritmos Bubble Sort o Selection Sort, y la multiplicación ingenua de matrices.

***O*(2N) - Exponencial:** El número de operaciones se duplica con cada elemento adicional. Estos algoritmos se vuelven inútiles (computacionalmente intratables) incluso con valores de *N* muy pequeños (ej. N=50).

# La Notación Big O (“O grande”)

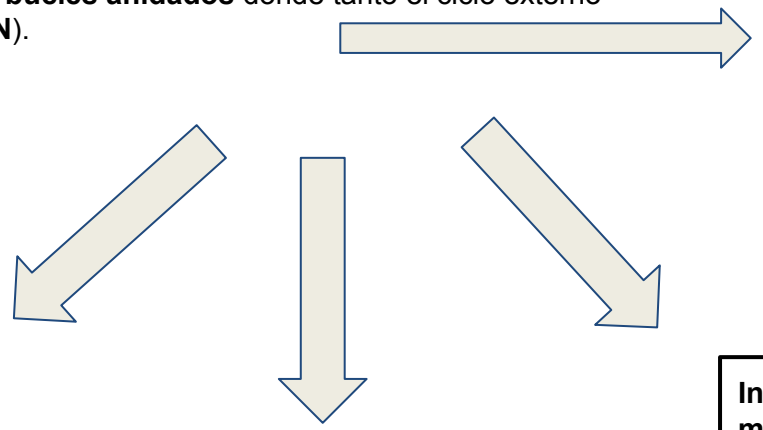
Ejemplo para complejidad temporal



# Algoritmos de ordenamiento: Bubble, Selection y Insertion

La complejidad temporal $O(N^2)$ en Bubble Sort, Selection Sort e Insertion Sort surge de una característica estructural que todos comparten: el uso de **bucles anidados** donde tanto el ciclo externo como el interno dependen de la cantidad total de datos (**N**).

Si lo traduces mentalmente a código en C, esto equivale invariablemente a tener un for (int i = 0; i < N; i++) que contiene por dentro otro bucle iterando sobre j (el cual también escala en función de "***N***").



**Bubble Sort (El de los intercambios constantes):** El bucle externo controla el número de "pasadas" por el arreglo (hasta "*N*" pasadas). El bucle interno recorre los elementos para intercambiar los adyacentes si están desordenados (hasta "*N-1*" comparaciones). En el peor de los casos (un arreglo ordenado a la inversa), el elemento más grande debe "burbujear" intercambiándose "*N-1*" veces, el segundo "*N-2*" veces, y así sucesivamente. Terminas multiplicando iteraciones por iteraciones.

**Selection Sort (El buscador del mínimo):** El bucle externo avanza lentamente, fijando la posición que se va a ordenar en ese paso (avanza "*N*" veces). El bucle interno tiene que escanear **todo** el resto del arreglo desordenado para encontrar el valor mínimo absoluto. Invariablemente, para colocar el primer elemento hace "*N-1*" comparaciones, para el segundo "*N-2*", etc. A diferencia de otros, Selection Sort *siempre* hace $O(N^2)$ operaciones, incluso si el arreglo ya estaba perfectamente ordenado.

**Insertion Sort (El de las cartas en la mano):** El bucle externo toma un elemento nuevo del arreglo (*N* veces). El bucle interno lo compara hacia atrás contra la sublista que ya está ordenada, desplazando valores en memoria para hacerle lugar. Si el arreglo está totalmente invertido, cada elemento nuevo obliga a desplazar hacia la derecha a todos los elementos anteriores. Sin embargo, su gran ventaja práctica es que si el arreglo ya está ordenado (o casi ordenado), el bucle interno se detiene de inmediato, dándole un rendimiento de $O(N)$ en el mejor de los casos.

# Algoritmos de ordenamiento : Bubble, selection y Insertion

Ejemplos de los 3 casos: ORDENAMIENTO DE UN ARREGLO

Bubble Sort

```
void bubbleSort(int arr[], int n) {
    for (int i = 0; i < n - 1; i++) {
        int swapped = 0;

        for (int j = 0; j < n - 1 - i; j++) {
            if (arr[j] > arr[j + 1]) {
                // Intercambio
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
                swapped = 1;
            }
        }

        // Si no hubo intercambios, el arreglo
ya está ordenado
        if (!swapped) break;
    }
}
```

Selection Sort

```
void selectionSort(int arr[], int n) {
    for (int i = 0; i < n - 1; i++) {
        int min_idx = i;

        // Buscar el índice del elemento más
pequeño en la parte desordenada
        for (int j = i + 1; j < n; j++) {
            if (arr[j] < arr[min_idx]) {
                min_idx = j;
            }
        }

        // Intercambiar el mínimo encontrado
con el primer elemento desordenado
        if (min_idx != i) {
            int temp = arr[i];
            arr[i] = arr[min_idx];
            arr[min_idx] = temp;
        }
    }
}
```

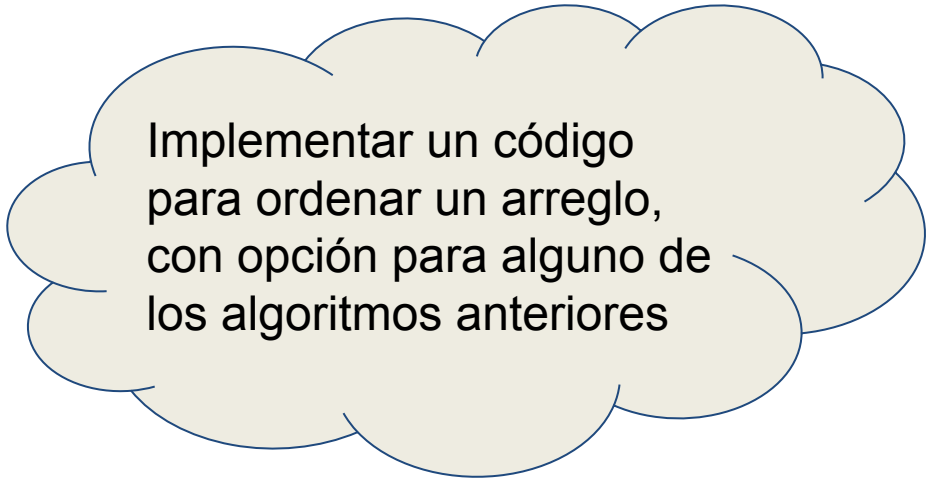
Insertion Sort

```
void insertionSort(int arr[], int n) {
    for (int i = 1; i < n; i++) {
        int key = arr[i];
        int j = i - 1;

        // Desplazar a la derecha los
elementos que sean mayores que 'key'
        while (j >= 0 && arr[j] > key) {
            arr[j + 1] = arr[j];
            j = j - 1;
        }

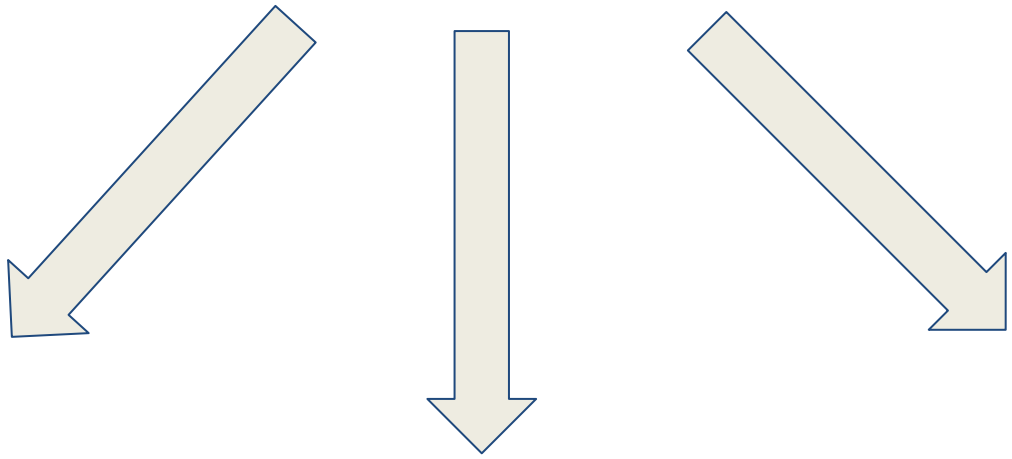
        // Insertar 'key' en su posición
matemáticamente correcta
        arr[j + 1] = key;
    }
}
```

# Algoritmos de ordenamiento : Bubble, mergeSort y Insertion



# Algoritmos de ordenamiento: Quicksort, mergeSort y ~~heapsort~~

si el objetivo es tomar datos caóticos y ordenarlos de la forma más rápida y óptima posible, estos tres son los que se utilizan en la industria.




Elige un elemento como "pivote" y arroja los menores a su izquierda y los mayores a su derecha. Luego repite esto con cada lado. Es extremadamente rápido en la práctica y usa muy poca memoria extra, aunque en su peor escenario puede caer a O(N2)

Divide el arreglo literalmente por la mitad una y otra vez hasta que quedan elementos sueltos. Luego, los empieza a "fusionar" (merge) de a pares, pero ya en el orden correcto. Su tiempo siempre es O(N log N), sin excepciones, pero a cambio requiere mucha memoria extra O(N) de complejidad espacial) para hacer esas fusiones.

Primero organiza todos los datos en una estructura de árbol binario virtual (llamado *Heap* o montículo). Esta estructura hace que el número más grande flote automáticamente a la raíz. El algoritmo saca esa raíz, la pone al final de la lista, y deja que el árbol se reacomode. Su ventaja es que garantiza el tiempo O(N log N) siempre, consumiendo memoria extra O(1).

# Algoritmos de ordenamiento: algoritmo quicksort!

```c
#include <stdio.h>

void swap(int* a, int* b) { //Función auxiliar para intercambiar variables en memoria
    int temp = *a;
    *a = *b;
    *b = temp;
}

int partition(int arr[], int low, int high) {//La lógica de partición
    int pivot = arr[high];                    // Tomamos el último elemento como pivote
    int i = (low - 1);                // Índice para ubicar los elementos menores al pivote

    // Recorremos el sub-arreglo actual
    for (int j = low; j <= high - 1; j++) {
        // Si el elemento actual es menor o igual al pivote...
        if (arr[j] <= pivot) {
            i++; // ...hacemos espacio...
            swap(&arr[i], &arr[j]); // ...y lo arrojamos a la izquierda.
        }
    }
    // Finalmente, colocamos el pivote en su posición central definitiva
    swap(&arr[i + 1], &arr[high]);

    // Devolvemos la posición final del pivote para dividir el arreglo
    return (i + 1);
}
```

```c
//La lógica recursiva
void quickSort(int arr[], int low, int high)
{
    // Caso base: si low >= high, el sub-arreglo tiene 1 o 0 elementos (ya está ordenado)
    if (low < high) {
        // Obtenemos el índice donde quedó el pivote anclado
        int pi = partition(arr, low, high);

        // Llamada recursiva para la mitad izquierda (menores al pivote)
        quickSort(arr, low, pi - 1);

        // Llamada recursiva para la mitad derecha (mayores al pivote)
        quickSort(arr, pi + 1, high);
    }
}

// --- Ejemplo de uso ---
int main() {
    int arr[] = {40, 10, 50, 30}; //Ejemplo
    int n = sizeof(arr) / sizeof(arr[0]);

    printf("Arreglo original: ");
    for(int i=0; i<n; i++) printf("%d ", arr[i]);

    quickSort(arr, 0, n - 1); // Llamamos a Quicksort enviando el índice inicial (0) y el final (n-1)

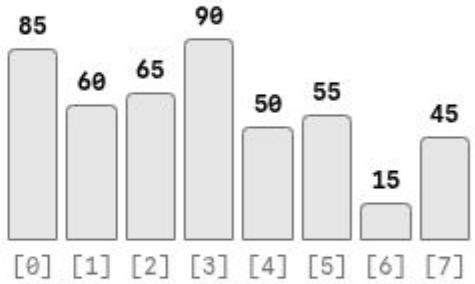
    printf("\nArreglo ordenado: ");
    for(int i=0; i<n; i++) printf("%d ", arr[i]);
    return 0;
}
```

# Algoritmos de ordenamiento: Ejemplo Quicksort!

CÓDIGO FUENTE EN C — PARTITION.C

```c
int partition(int arr[], int low, int high) {
  int pivot = arr[high]; // Elegir pivote
  int i = (low - 1);   // Índice de menores
  for (int j = low; j < high; j++) {
    if (arr[j] <= pivot) { // Comparación
      i++;
      swap(&arr[i], &arr[j]); // Intercambio
    }
  }
  swap(&arr[i + 1], &arr[high]); // Ubicar pivote
  return (i + 1);
}
```

ESTADO DEL ARREGLO — completo




Ver recurso

# Algoritmos de ordenamiento: Ejercicio Quicksort!

Crear un programa en C que defina una estructura para un "Paquete de Telemetría", inicialice un arreglo con datos desordenados y lo ordene de **mayor a menor** basándose en la altitud, utilizando el algoritmo Quicksort.

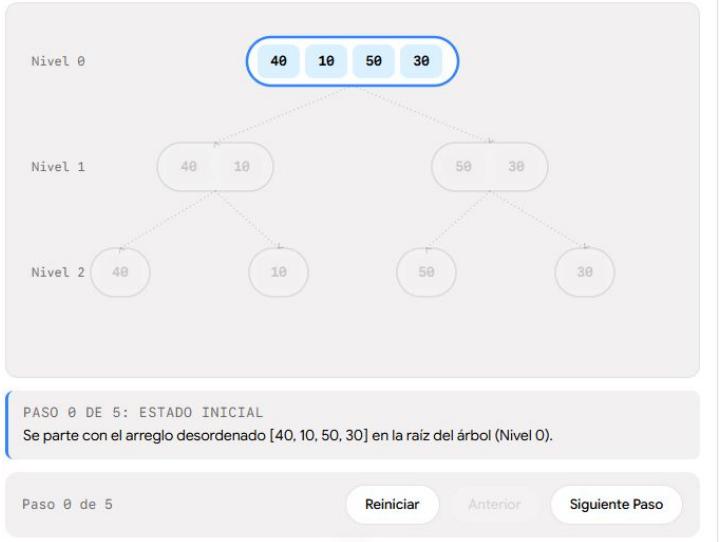
**Requerimientos del código:**

1. **Estructuras:** Define un `struct Telemetria` que contenga:
   - `int id_paquete` (Un número correlativo).
   - `float altitud` (El valor por el cual deben ordenar).
   - `float temperatura`.
2. **Modularización:** El código debe estar separado. El algoritmo Quicksort debe estar en un archivo `ordenamiento.c` con sus prototipos en `ordenamiento.h`, mientras que el `main.c` solo debe encargarse de inicializar los datos y mostrar los resultados.
3. **Lógica del Quicksort:** Adapta el algoritmo Quicksort (que vimos para enteros) para que ordene estructuras completas basándose *únicamente* en el campo `altitud` de forma descendente (de mayor a menor).

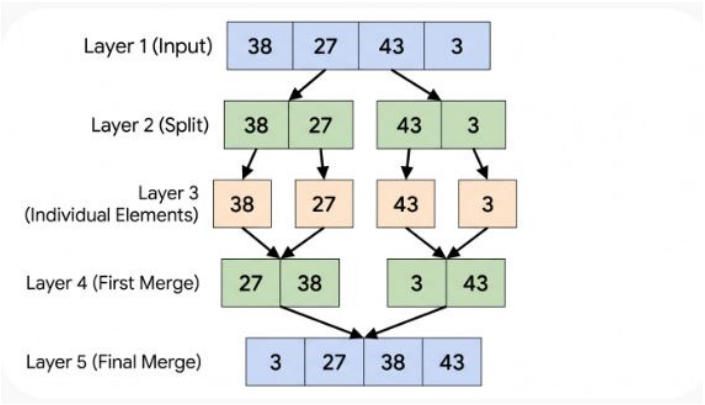
Telemetria datos[] = { {101, 1500.5, 22.1}, {102, 3200.0, -5.4}, {103, 800.2, 18.0}, {104, 4100.8, -12.3}, {105, 2100.0, 5.5} };

# Algoritmos de ordenamiento: Mergesort

**Fase 1:** Dividir (The Split): Mergesort toma el arreglo original y lo corta exactamente por la mitad, sin importarle qué números hay adentro ni intentar compararlos. Luego toma esas dos mitades y las vuelve a cortar por la mitad.
Hace esto recursivamente hasta que el arreglo queda despedazado en "sub-arreglos" de un solo elemento.
**Fase 2:** El algoritmo empieza a retroceder en la recursión, tomando de a dos esos mini-arreglos para fusionarlos en uno nuevo, más grande, pero asegurándose de que los elementos entren en el orden correcto.




Ver recurso



# Algoritmos de ordenamiento: Mergesort

```c
#include <stdio.h>
#include <stdlib.h>

// Función para fusionar dos mitades ordenadas de un arreglo
void merge(int arr[], int left, int mid, int right) {
    int i, j, k;
    int n1 = mid - left + 1; // Tamaño de la mitad izquierda
    int n2 = right - mid;    // Tamaño de la mitad derecha

    // Crear arreglos temporales
    int L[n1], R[n2];

    // Copiar datos a los arreglos temporales L[] y R[]
    for (i = 0; i < n1; i++)
        L[i] = arr[left + i];
    for (j = 0; j < n2; j++)
        R[j] = arr[mid + 1 + j];

    // Fusionar los arreglos temporales de vuelta en arr[left..right]
    i = 0; // Índice inicial del primer subarreglo
    j = 0; // Índice inicial del segundo subarreglo
    k = left; // Índice inicial del arreglo fusionado

    while (i < n1 && j < n2) {
        if (L[i] <= R[j]) {
            arr[k] = L[i];
            i++;
        } else {
            arr[k] = R[j];
            j++;
        }
        k++;
    }
    while (i < n1) {    // Copiar los elementos restantes de L[], si los hay
        arr[k] = L[i];
        i++;
        k++;
    }
    while (j < n2) {    // Copiar los elementos restantes de R[], si los hay
        arr[k] = R[j];
        j++;
        k++;
    }
}
void mergeSort(int arr[], int left, int right) {// Función principal de Merge Sort que divide el arreglo
    if (left < right) {
        int mid = left + (right - left) / 2;       // Encontrar el punto medio para dividir el arreglo en dos mitades
        mergeSort(arr, left, mid);       // Llamada recursiva para la primera y segunda mitad
        mergeSort(arr, mid + 1, right);
        merge(arr, left, mid, right);        // Fusionar las dos mitades ordenadas
    }
}

void printArray(int arr[], int size) {
    for (int i = 0; i < size; i++)
        printf("%d ", arr[i]);
    printf("\n");
}
int main() {
    int arr[] = {38, 27, 43, 3, 9, 82, 10};
    int arr_size = sizeof(arr) / sizeof(arr[0]);
    printArray(arr, arr_size);
    mergeSort(arr, 0, arr_size - 1);    // Llamar a la función de ordenamiento
    printf("\nArreglo ordenado: \n");
    printArray(arr, arr_size);
    return 0;
}
```

# Algoritmos de ordenamiento: librería en C para quick sort (uso)

```c
#include <stdio.h>
#include <stdlib.h> // Requerido para qsort

// 1. Definir la función comparadora
// qsort requiere que devuelva:
// un número < 0 si 'a' debe ir antes que 'b'
// 0 si 'a' y 'b' son iguales
// un número > 0 si 'a' debe ir después que 'b'
int comparar(const void *a, const void *b) {
    // Hacemos cast de los punteros void* a int* y extraemos su valor
    int valorA = *(const int *)a;
    int valorB = *(const int *)b;

    // Forma segura de comparar evitando desbordamientos de enteros (overflow)
    if (valorA < valorB) return -1;
    if (valorA > valorB) return 1;
    return 0;

    // Nota: Si quisieras orden descendente, simplemente invierte la lógica:
    // si (valorA < valorB) return 1; ...
}

// Función auxiliar para imprimir
void imprimirArreglo(int arr[], int n) {
    for (int i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");
}

int main() {
    int arr[] = {38, 27, 43, 3, 9, 82, 10};
    int n = sizeof(arr) / sizeof(arr[0]);

    printf("Arreglo original: \n");
    imprimirArreglo(arr, n);

    // 2. Ejecutar qsort
    // Parámetros:
    // - Puntero al arreglo
    // - Cantidad de elementos
    // - Tamaño en bytes de cada elemento
    // - Puntero a la función comparadora
    qsort(arr, n, sizeof(int), comparar);

    printf("Arreglo ordenado: \n");
    imprimirArreglo(arr, n);

    return 0;
}
```